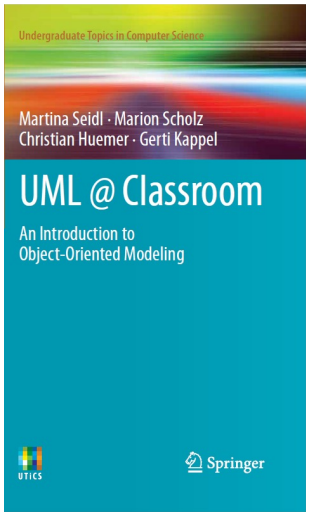
Business Informatics Group

Vienna University of Technology



Object-Oriented Modeling

State Machine Diagram

Slides accompanying UML@Classroom
Version 1.0

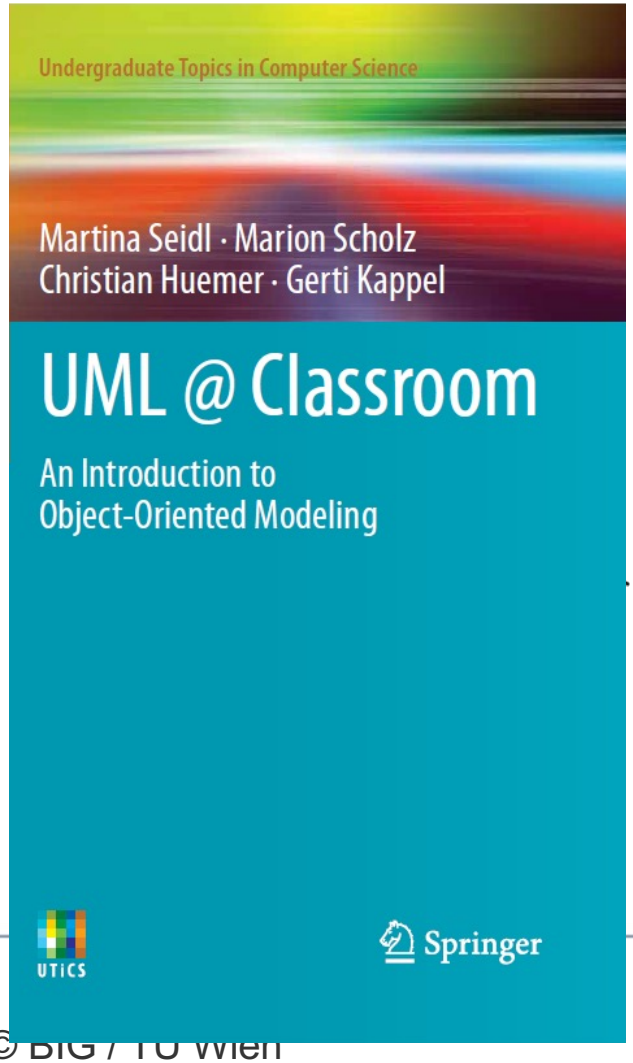
Business Informatics Group

Institute of Software Technology and Interactive Systems
Vienna University of Technology

Favoritenstraße 9-11/188-3, 1040 Vienna, Austria
phone: +43 (1) 58801-18804 (secretary), fax: +43 (1) 58801-18896
office@big.tuwien.ac.at, www.big.tuwien.ac.at

Literature

- The lecture is based on the following book:



UML @ Classroom:
An Introduction to Object-Oriented Modeling
Martina Seidl, Marion Scholz, Christian Huemer and Gerti Kappel

Springer Publishing, 2015

ISBN 3319127411

- Use Case Diagram
- Structure Modeling
- **State Machine Diagram**
- Sequence Diagram
- Activity Diagram

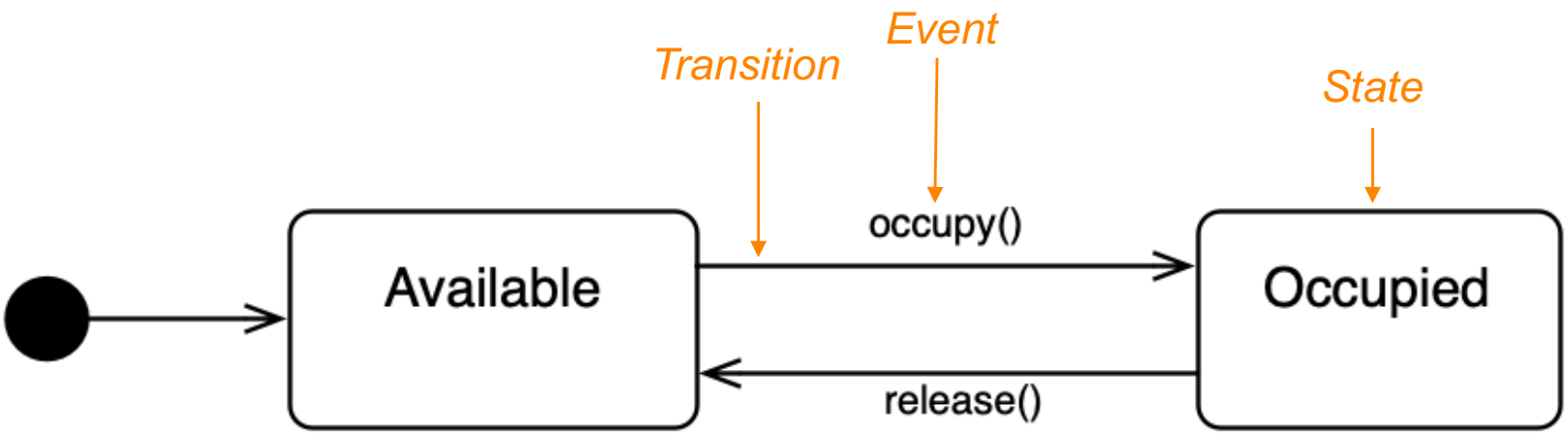
Content

- Introduction
- States
- Transitions
- Types of events
- Types of states
- Entry and exit points

Introduction

- Every object takes a finite number of different states during its life
- State machine diagram is used as follows:
  - To model the possible states of a system or object
  - To show how state transitions occur as a consequence of events
  - To show what behavior the system or object exhibits in each state

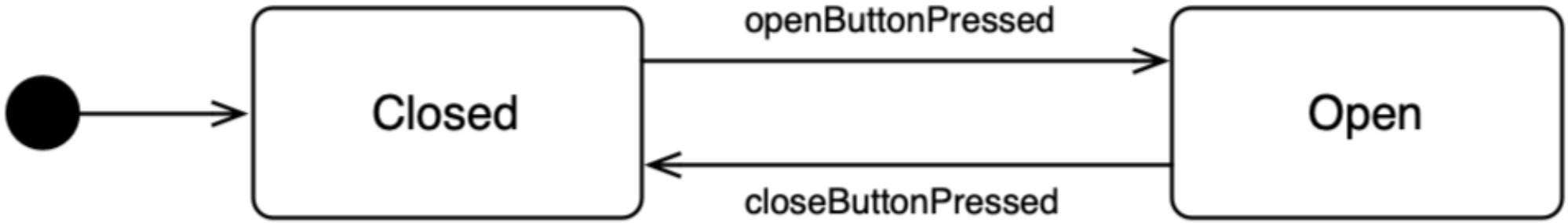
- Example: high-level description of the behavior of a **Room** object



Some examples to get an idea of what it's all about …

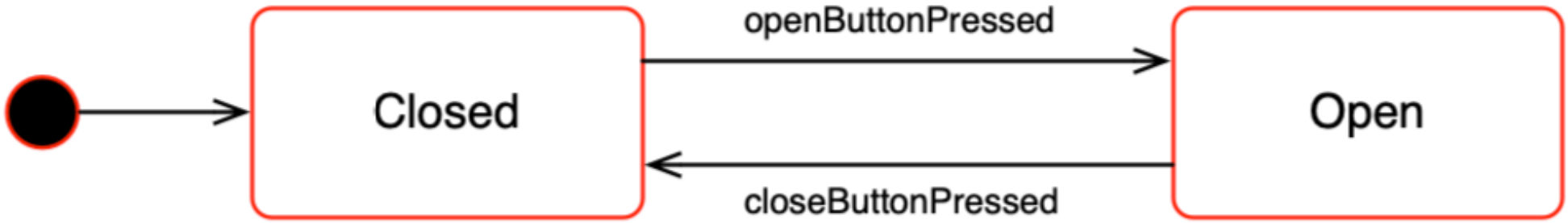
- The following are some simple State Machine Diagrams
  - To help get idea about core concepts/components of SM Diagrams
  - We'll then go through each part of a diagram step-by-step …

A very simple State Machine Diagram



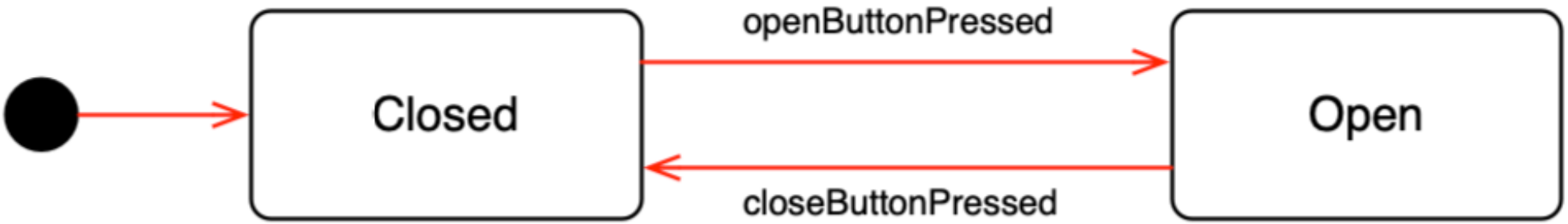
States

- There are 3 states here
  - Initial (the black circle – a special case – a “pseudostate”)
  - Closed
  - Open



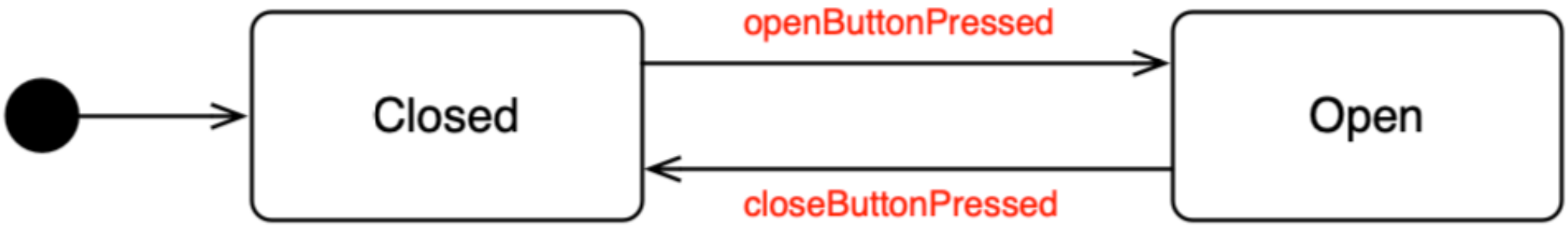
Transitions

- There are 3 transition (arrows indicating change to new state)
  - *from* **INITIAL** *to* **CLOSED**
  - *from* **CLOSED** *to* **OPEN**
  - *from* **OPEN** *to* **CLOSED**

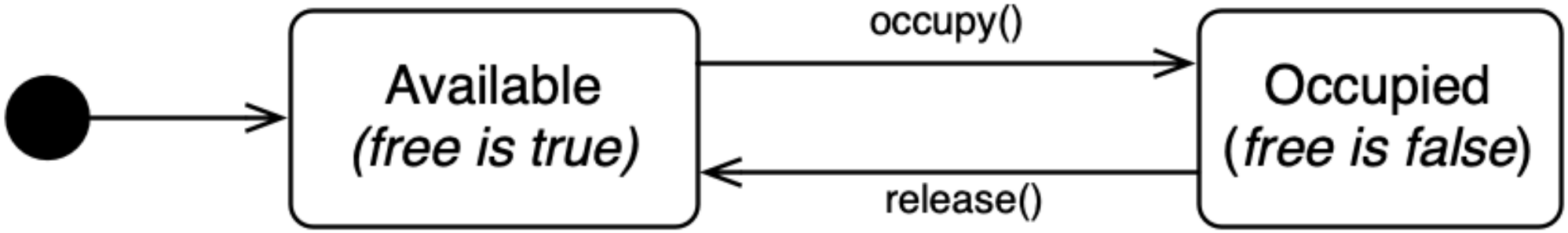


Events

- There are 2 events
  - openButtonPressed
  - closedButtonPressed

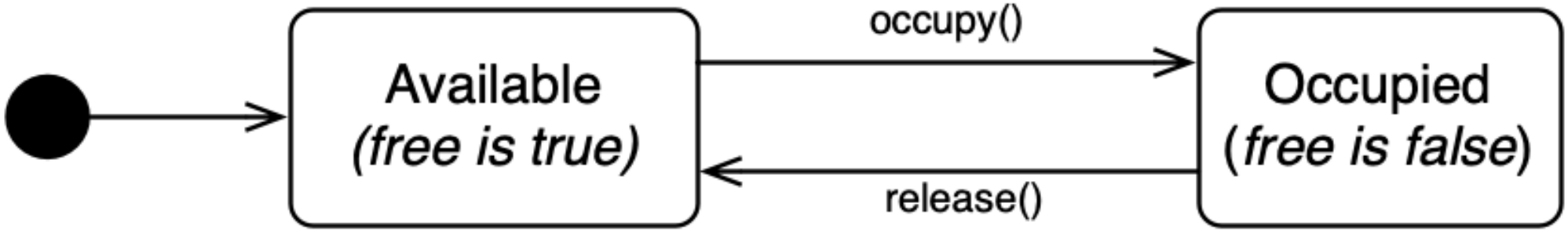


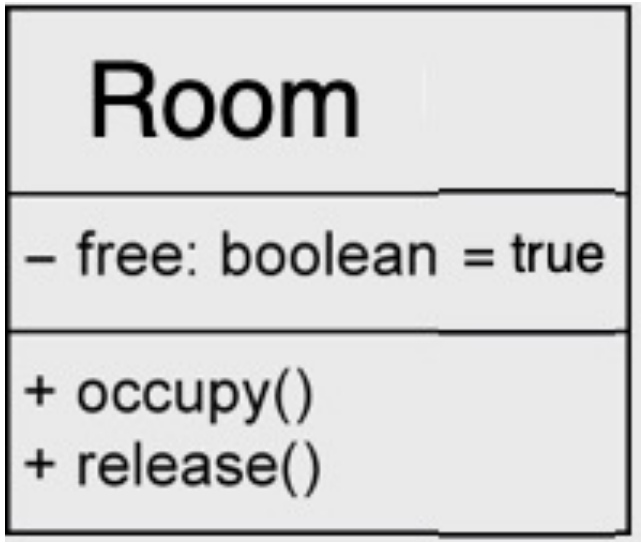
Example: Room (e.g. bathroom – occupied or not)
- an event can be a operation invocation, e.g.: `occupy()`



© BIG / TU Wien

Here is class diagram and Java implementation ..





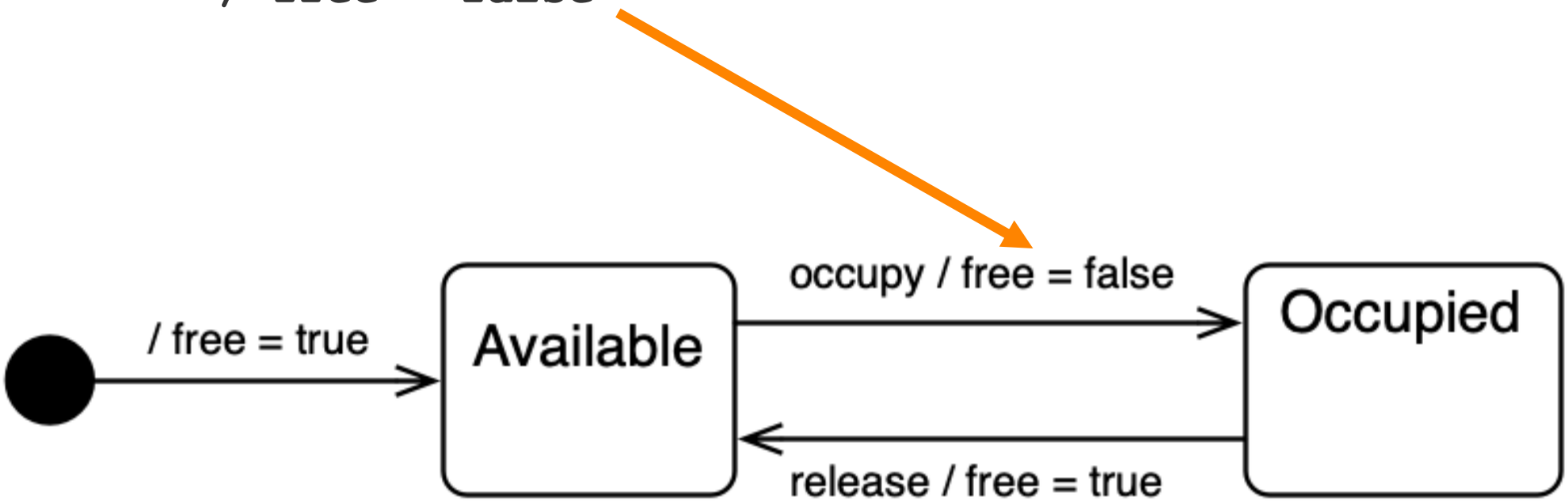

```
class Room
{
   private boolean free = true;

   public void occupy()
   {
      free = false;
   }

   public void release()
   {
      free = true;
   }
}
```

© BIG / TU Wien

Like all UML diagrams – we can choose level of abstraction / detail - we can indicate variable change details as behaviours in diagram

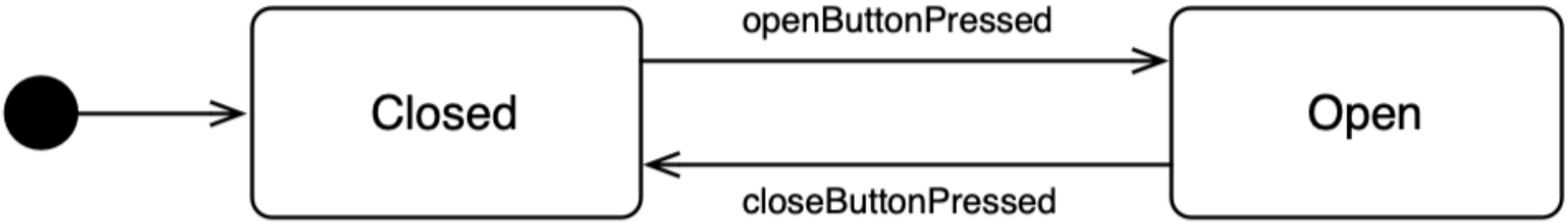
- We can indicate BEHAVIOURS by adding a forward slash "/" in text describing the Transition
  - E.g. for the Transition from state AVAILABLE to state OCCUPIED we can indicate that there is some kind of **occupy** event message, and that we want variable **`free`** to be set to **`false`** by writing

```
/ free = false
```



Lift Doors

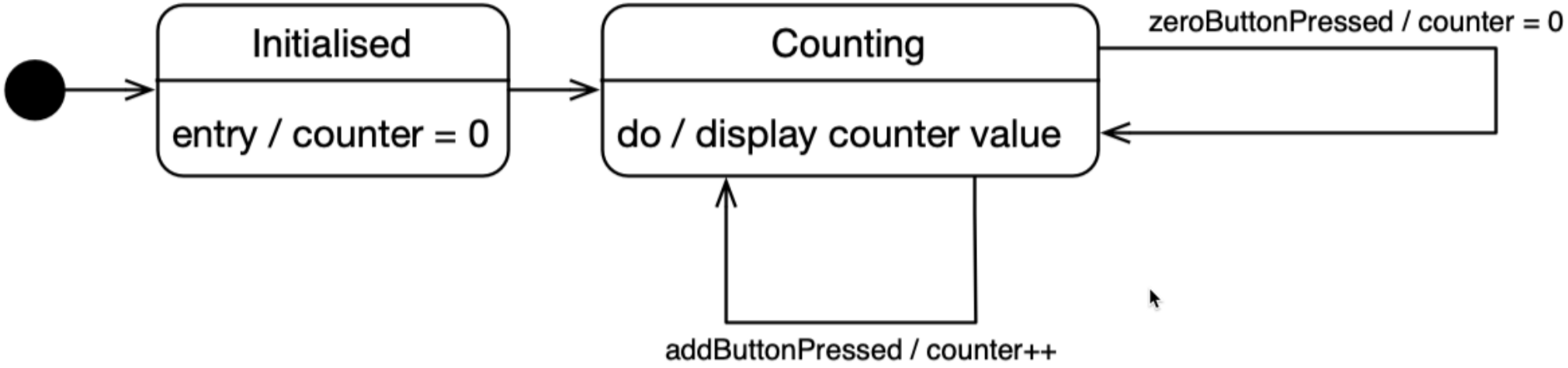
- 2 buttons: open & close
- Black circle is Initial State (where things start)



Digital click counter (e.g. counting passengers on plane)

- 2 buttons: zero & add
  - Events are: **zeroButtonPressed** and **addButtonPressed**
- Black circle is Initial State (where things start)

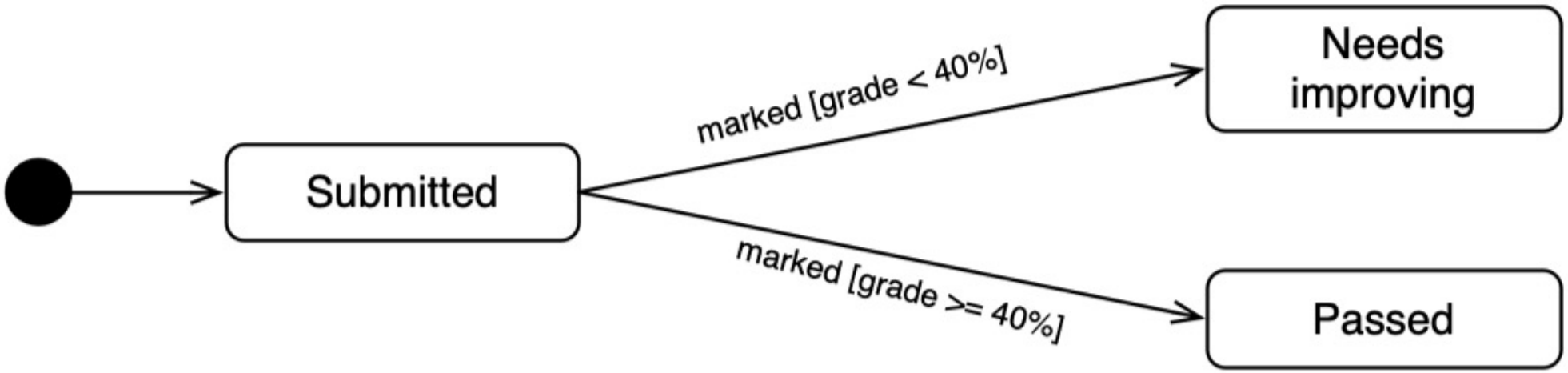
- Behaviours can change variables (e.g. `counter`) in the system



Example: Decision Node – different guard conditions after same event

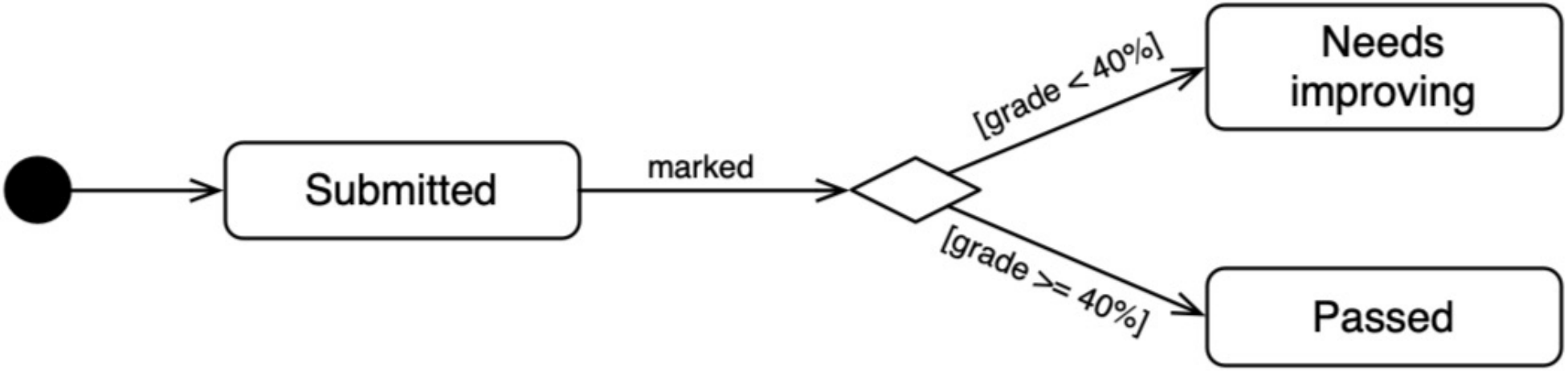
Student Essay (minimum pass is 40%)

- After event of being marked the essay object moves into either the PASSED state or NEEDS IMPROVING
- Based on value of **`grade`**
  - guard conditions in square brackets

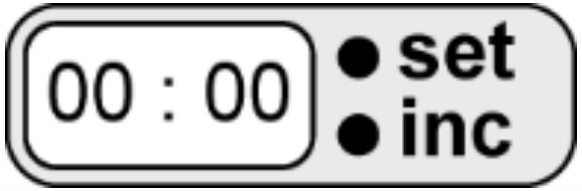


Example: Decision Node – different conditions after same event

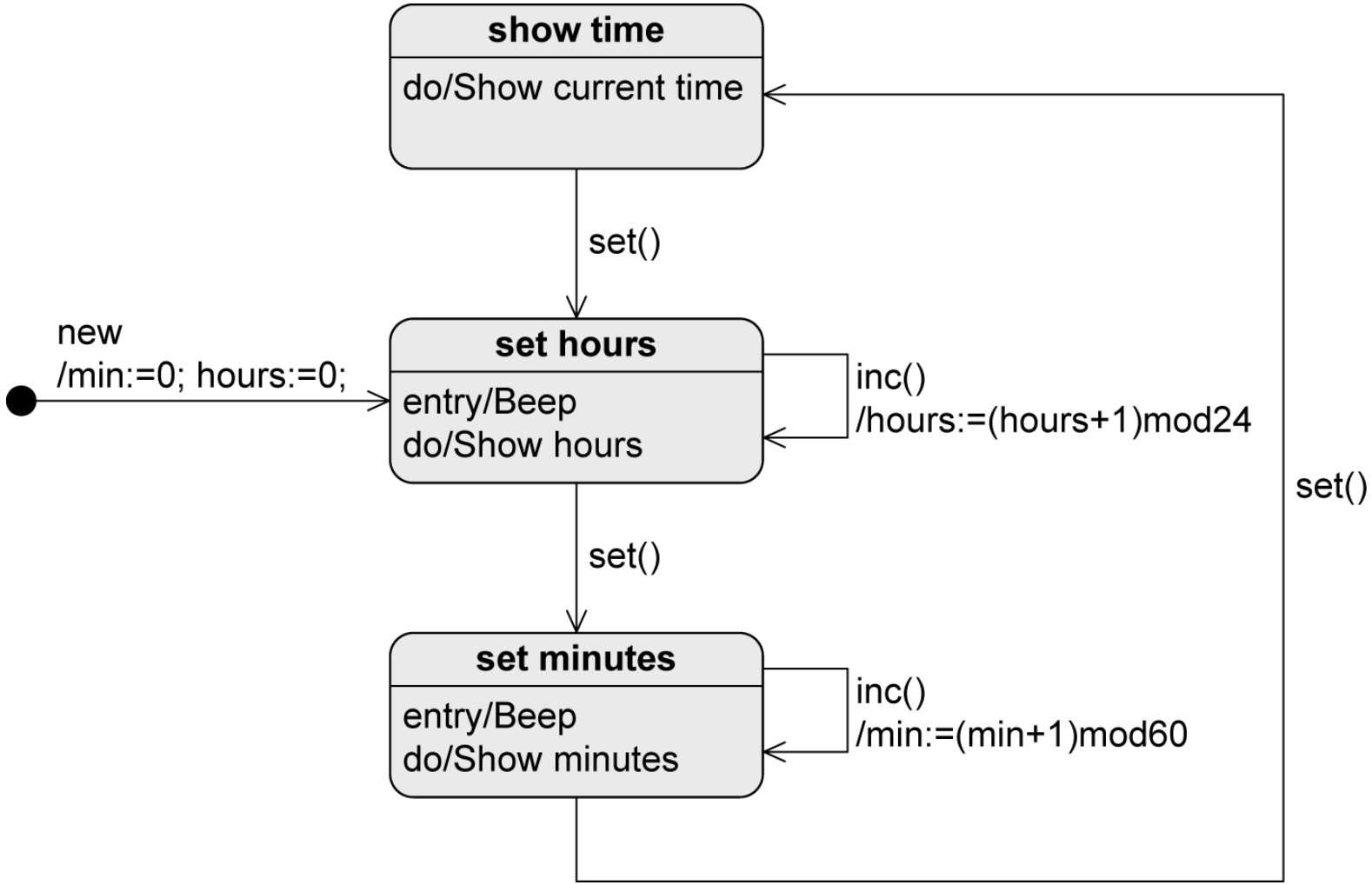
- Decision node is a notational convenience
  - So could create diagram without them
  - But when used well, they make simpler/easier to understand logic…



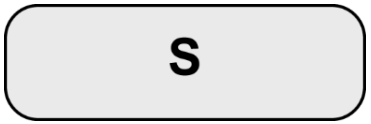
Example: Digital Clock
/entry and /do event behaviours



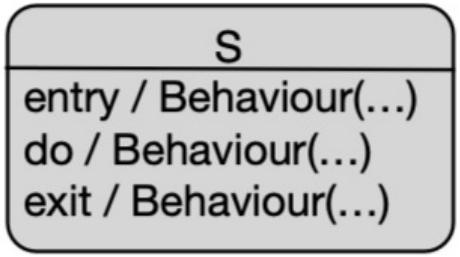
DigitalClock
– min: int – hours: int
+ set(): void + inc(): void



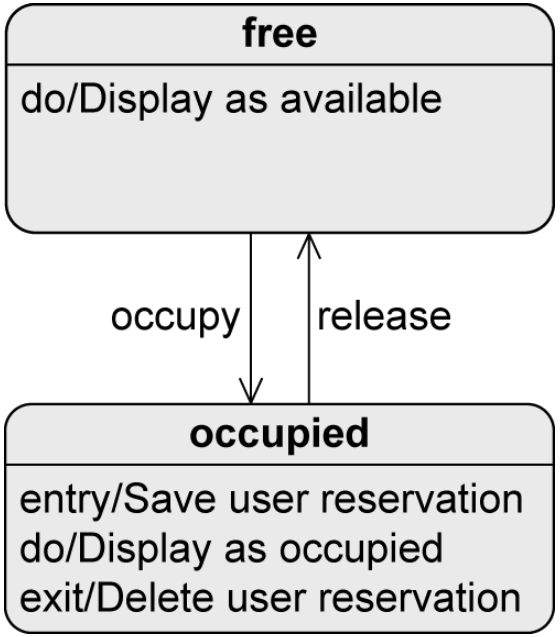
State



- States = nodes of the state machine

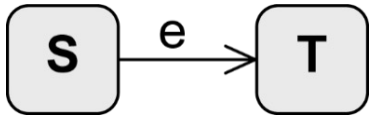


- When a state is active
  - The object is in that state
  - All internal behaviours specified in this state can be executed
    - A behaviour can consist of multiple actions

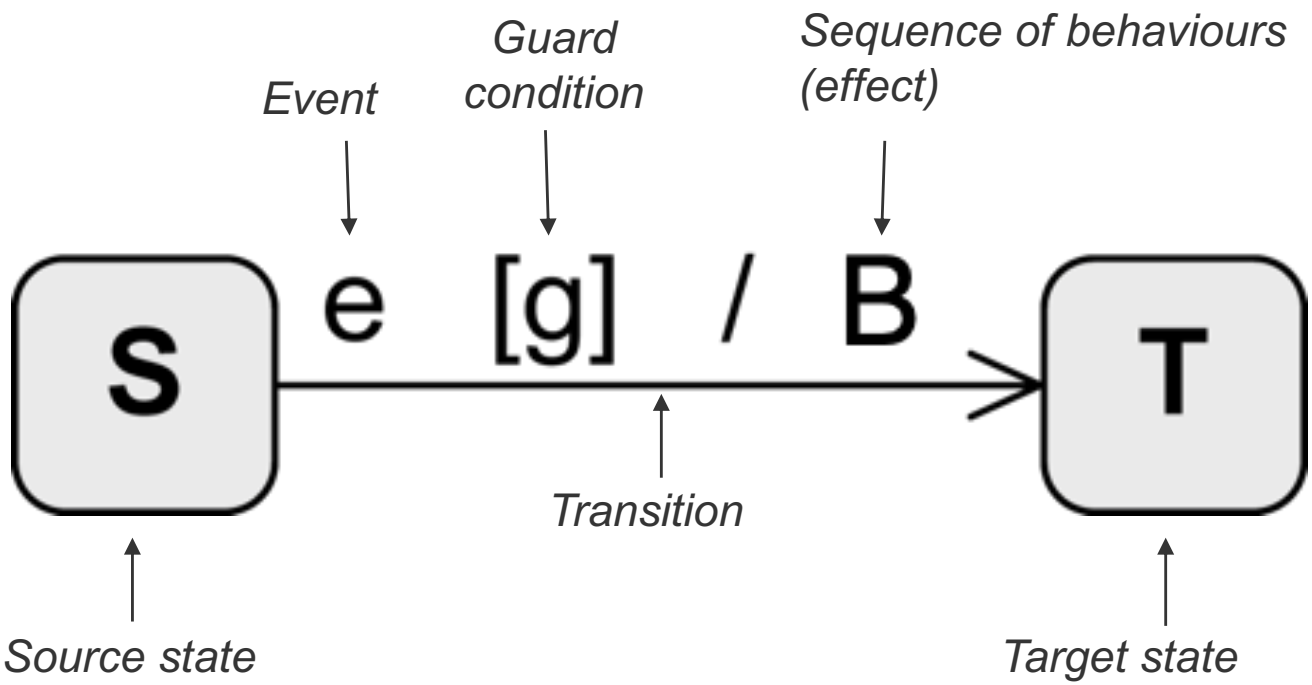


- entry / Behaviour(...)
  - Executed when the object enters the state
- exit / Behaviour(...)
  - Executed when the object exits the state
- do / Behaviour(...)
  - Executed while the object remains in this state

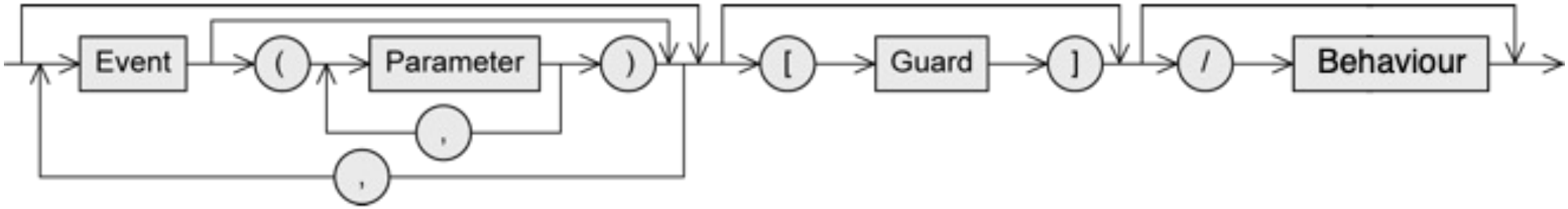
Transition



- Change from one state to another



Transition – Syntax



- Event (trigger)
  - Exogenous stimulus
  - Can trigger a state transition
- Guard (condition)
  - Boolean expression
  - If the event occurs, the guard is checked
  - If the guard is true
    - All behaviours in the current state are terminated
    - Any relevant exit behaviours is executed
    - The transition takes place
  - If the guard is false
    - No state transition takes place, the event is discarded
- Behaviour (effect)
  - Sequence of behaviours executed during the state transition

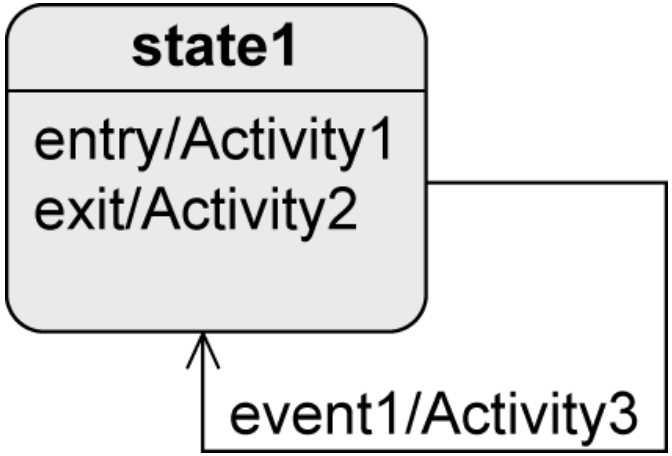
NOTE: Behaviours used to be called Activities – either term fine for this module

Transition – Types (1/3) – Internal / External transition

Internal transition

- If **`event1`** occurs
  - Object remains in **`state1`**
  - **`Activity3`** is executed

External transition



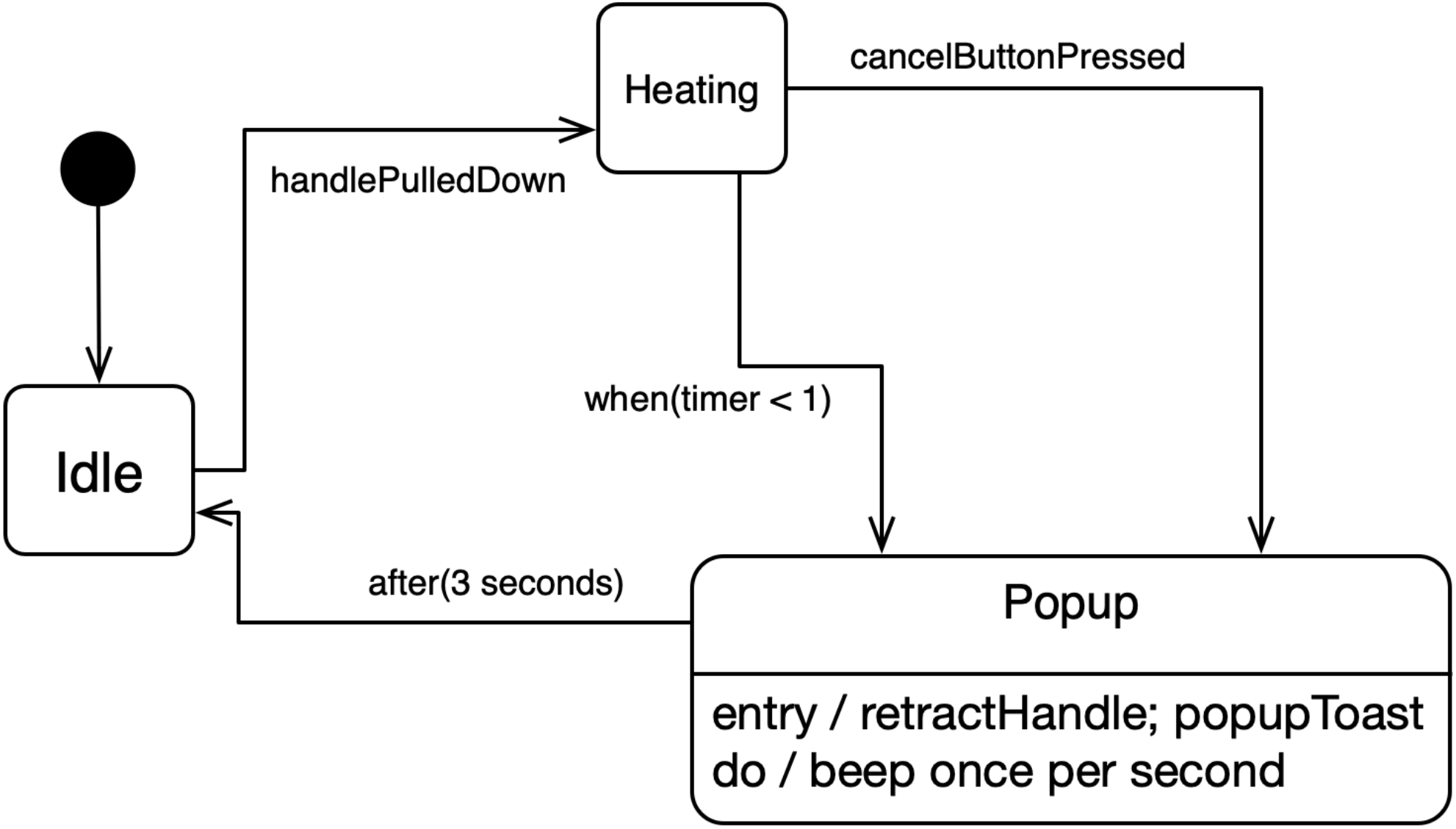
- If **`event1`** occurs
  - Object leaves **`state1`** and **`Activity2`** is executed
  - **`Activity3`** is executed
  - Object enters **`state1`** and **`Activity1`** is executed

Have a go at creating a diagram…
State Machine Diagram to model bread toaster system:

- The system starts in idle mode.
- In idle mode if the handle is pulled down the system moves into heating mode.
- In heating mode, if the timer reaches zero or if the cancel button is pressed then the system goes into popup mode.
- In popup mode the handle release is retracted, the toasted bread should popup and the system starts beeping once per second
  - after 3 seconds the system returns into idle mode.

- (1) identify STATES / (2) identify TRANSITIONS / (3) identify EVENTS
- (4) identify GUARD CONDITIONS / (5) identify BEHAVIOURS
- (6) – refine with entry/do/exit internal transitions, decision diamond etc.

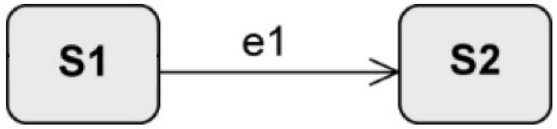
State Machine Diagram: Bread Toaster system



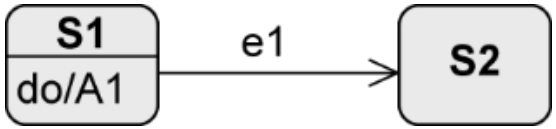
Transition – Types (2/3) – Event causing external transition

- An EVENT triggers transition from one state to another
  - If a ”do” action is being executed, then it gets aborted

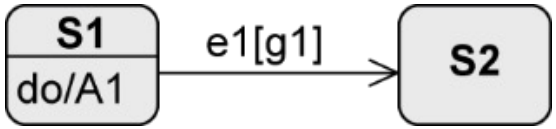
- When do the following transitions take place?



IF **e1** occurs
THEN object immediately changes to **S2**



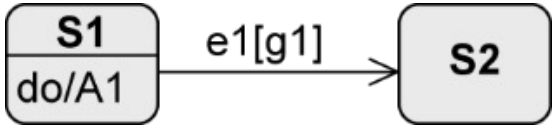
IF **e1** occurs
THEN **A1** is aborted and the object changes to **S2**



IF **e1** occurs AND **g1** evaluates to true
THEN **A1** is aborted and the object changes to **S2**

NOTE – if event e1 occurs at a later time, g1 will be tested again …

- When do the following transitions take place?



IF **e1** occurs AND **g1** evaluates to true
THEN **A1** is aborted and the object changes to **S2**

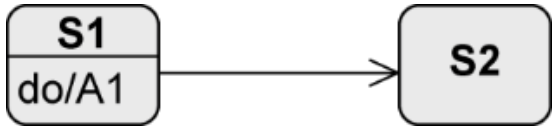
IF **e1** occurs AND **g1** evaluates to FALSE
THEN object remains in state **S1**
… HOWEVER, if event **e1** occurs at a later time, **g1** will be tested again …

So – IF system is in state **S1**,
EACH time event e1 occurs
THEN condition **g1** will be tested

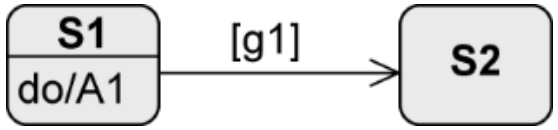
Transition – Types (3/3) – completion event (no external event) (so be careful if you have transitions with no external event…)

- When do the following transitions take place?

S2 immediately transitions to **S2**



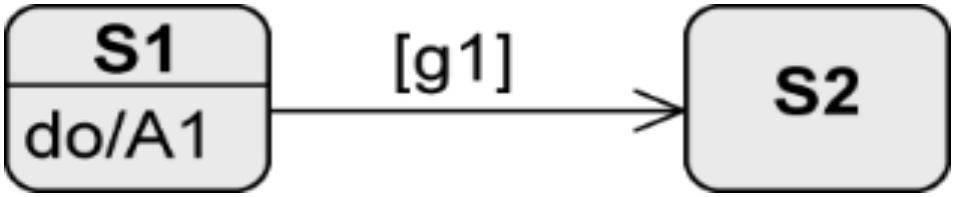
As soon as the execution of **A1** is finished, a completion event is generated that initiates the transition to **S2**



As soon as the execution of **A1** is finished, a completion event is generated;
At this time if **g1** evaluates to true, the transition takes place; If not, this transition can never happen (**g1** will not be tested for again)

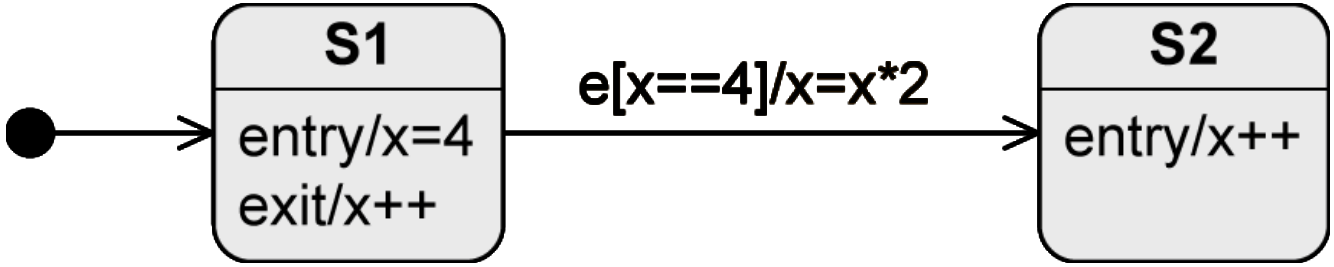
completion event (no external event) – AVOID if you can …

- So in general we try to AVOID this type of transition
  (unless that's the only/best way to model the system behaviour…)



Transition – Sequence of Activity Executions

- Assume **`S1`** is active … what is the value of **`x`** after **`e`** occurred?



`S1` becomes active, **`x`** is set to the value **`4`**

`e` occurs, the guard is checked and evaluates to true

`S1` is exited, **`x`** is set to **`5`**

The transition takes place, **`x`** is set to **`10`**

`S2` is entered, **`x`** is set to **`11`**

Event – Types (1/2)

- **Signal event (e.g. user interaction, message from external system)**
  Receipt of a signal
  - E.g., `rightmousedown`, `sendSMS(message)`

- **Call event (operation (method) being invoked)**
  Operation call – ALWAYS has parentheses – round brackets "(…)"
  - E.g., `occupy(user,lectureHall)`, `register(exam)`

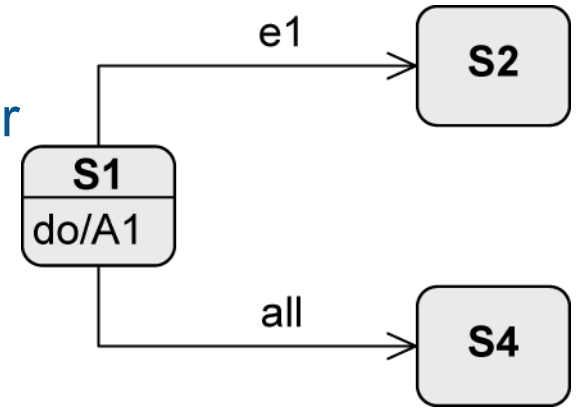
- **Time event**
  Time-based state transition
  - Relative: based on the time of the state being entered
    - E.g. `after(5 seconds)`
  - Absolute (special case of Change event involving time … see next slide)
    - E.g. `when(time==16:00)`, `when(date==20150101)`

Event – Types (2/2)

- **Any receive event (any other event…)**

  Occurs when any event occurs that does not trigger another transition from the active state

  - Keyword **`all`** (think “all other events”)



- **Completion event**

  Generated automatically when everything to be done in the current state is completed (e.g. after “`do/<behaviour>`” completed)

- **Change event (for example an absolute time event)**

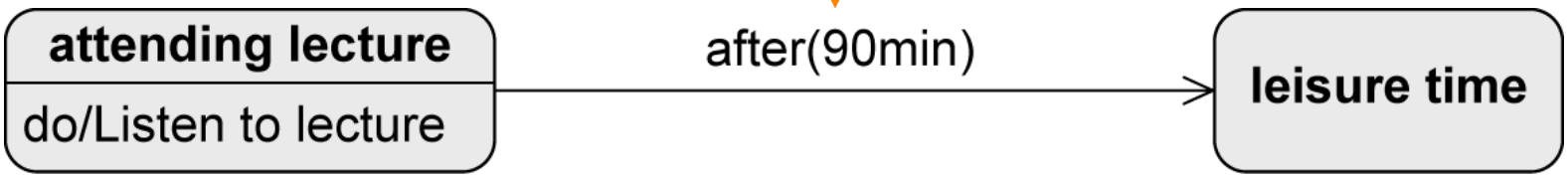
  Permanently checking whether a condition becomes true

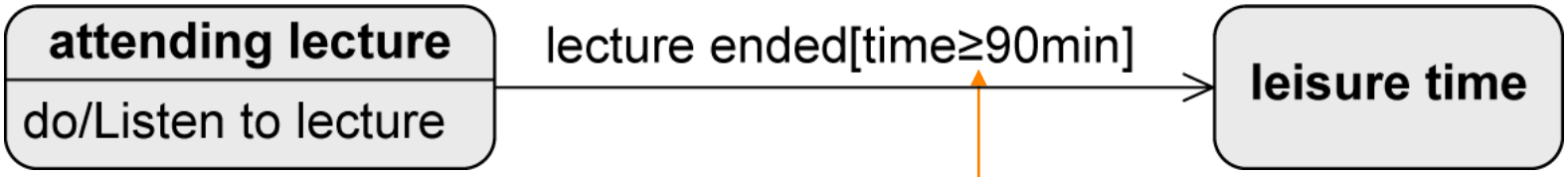
  - Keyword **`when (<Boolean condition>)`**
  - E.g. **`when(x > y)`**

    `when(time == 12:00)`

Change Event vs. Guard Condition
do NOT confuse them!

Checked regularly to see if is true
(e.g. every frame/clock tick)





Only checked once,
when event "lecture ended" occurs

Question: What if the lecture is shorter than 90min?

Initial State

- "Start" of a state machine diagram
- Pseudostate
  - Transient, i.e., system cannot remain in that state
  - Rather a control structure than a real state
- No incoming edges
- If >1 outgoing edges
  - Guards must be mutually exclusive and cover all possible cases to ensure that exactly one target state is reached
  - (try to have just one transition from the Initial state…)
- If initial state becomes active,
  the object immediately switches to the next state
  - No events allowed on the outgoing edges
    - exception: **`new()`**
      indicating object has just been created
  - May add initialization Behaviours (e.g. **`free = true`**)

Final State and Terminate Node

Final State

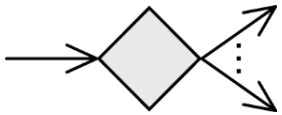
- Real state
- Marks the end of the sequence of states
- Object can remain in a final state forever

Terminate State

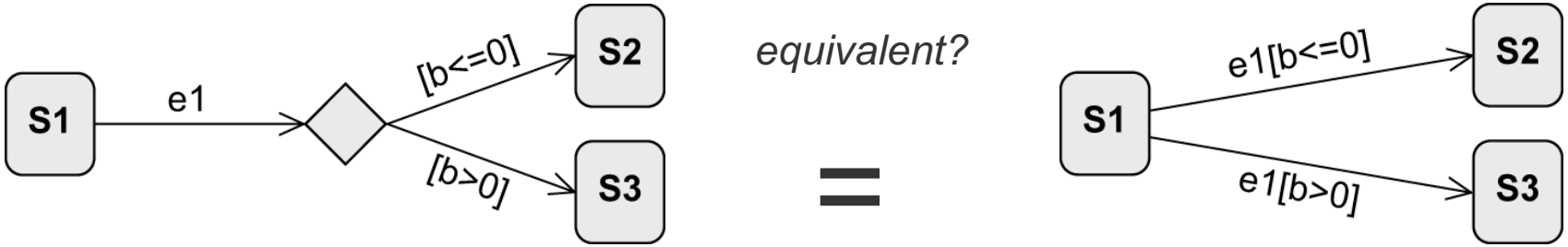
- Pseudostate
- Terminates the state machine
- The modeled object ceases to exist (= is deleted)

- Mnenomic: “Terminate means X-TERMINATE – delete object”

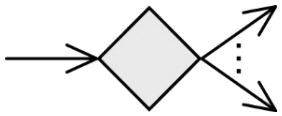
Decision Node



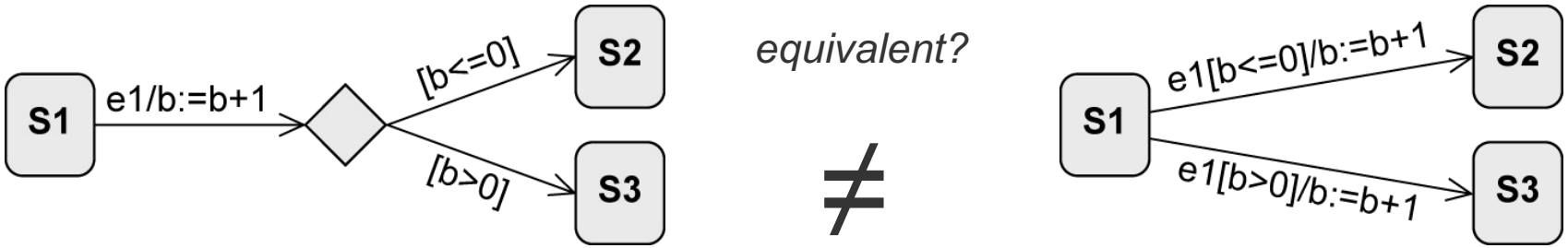
- Pseudostate
- Used to model alternative transitions



Decision Node



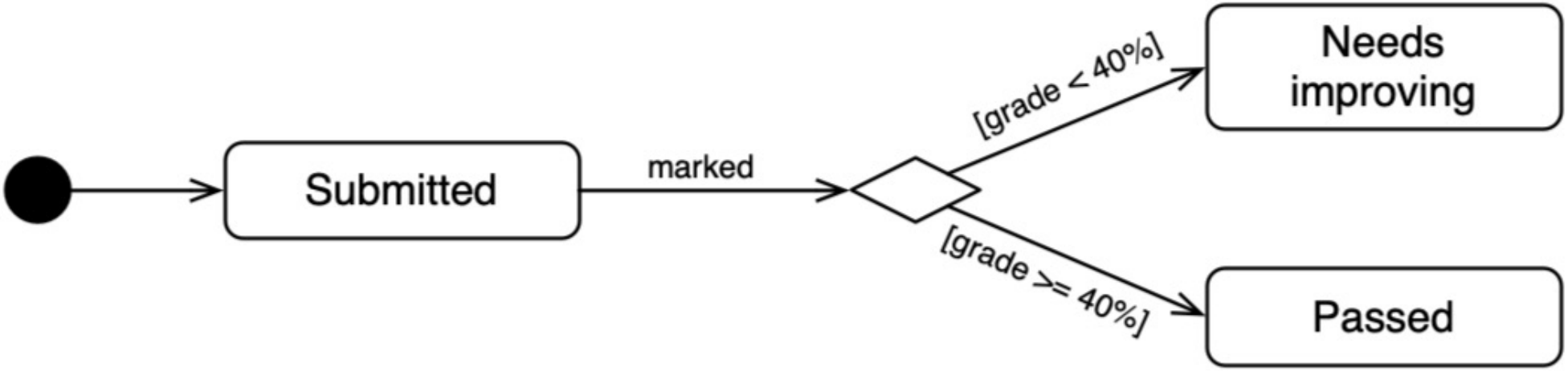
- Pseudostate
- Used to model alternative transitions



Example: Decision Node

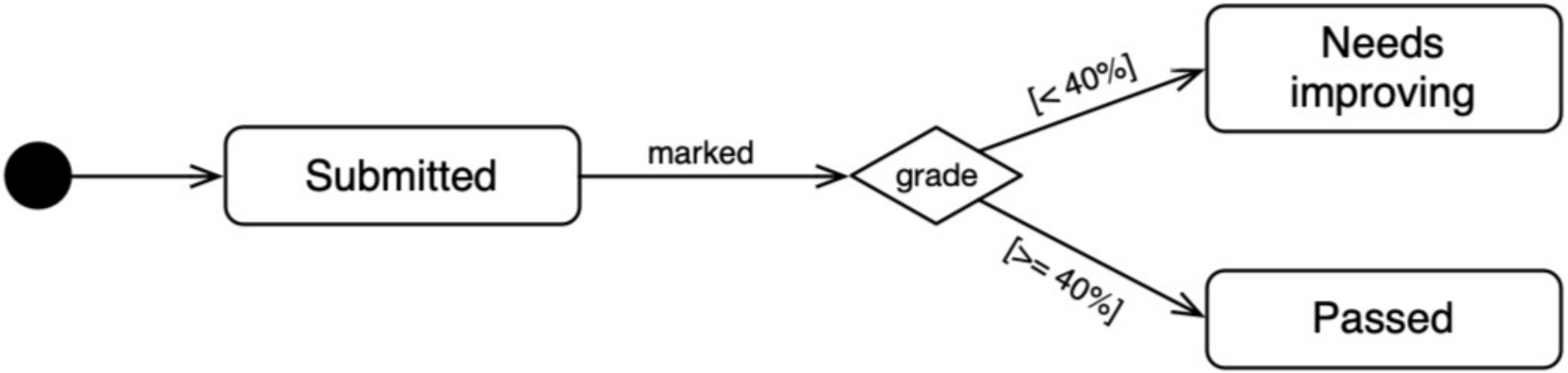
Student Essay (minimum pass is 40%)

- After being marked the essay object moves into either the “Passed” state or “Needs Improving”
- Based on value of “grade”



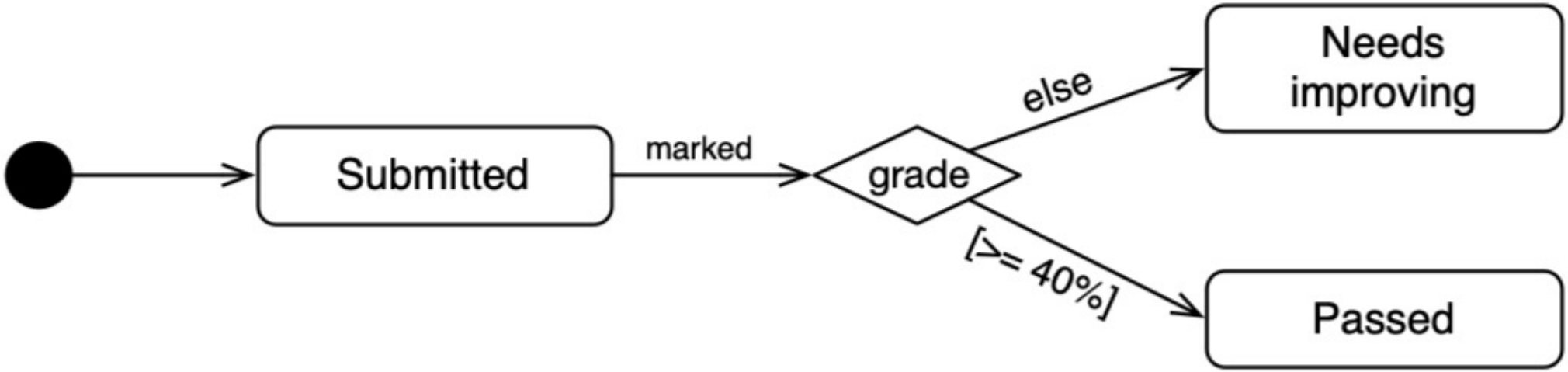
Variable to be tested INSIDE decision diamond

- When all guard conditions related to values of the same variable we can name the variable INSIDE the diamond simplifying the guard conditions to the operators



ELSE – ensure mutually exclusive guard conditions

- To ensure one transition from the decision diamond will always be selected, we can use the "**else**" keyword to indicate that this transition is to be followed if none of the other transition's guard conditions were true



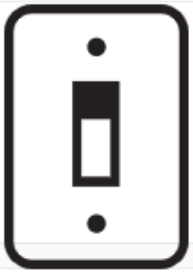
State Chart (machine!) XML
(State Machines used to be called State Charts…)

- Standard for States-Transitions as XML developed
  - SCXML – State Chart XML
  - Visit interactive State Charts(machines!) at
  - https://scion.scxml.io/tutorials/introduction

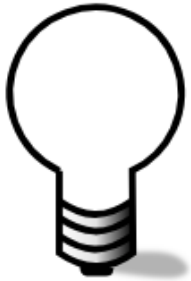
Example: a light switch

Let's say you have a light switch. Whenever you touch it, it turns on if it was off and off if it was on.

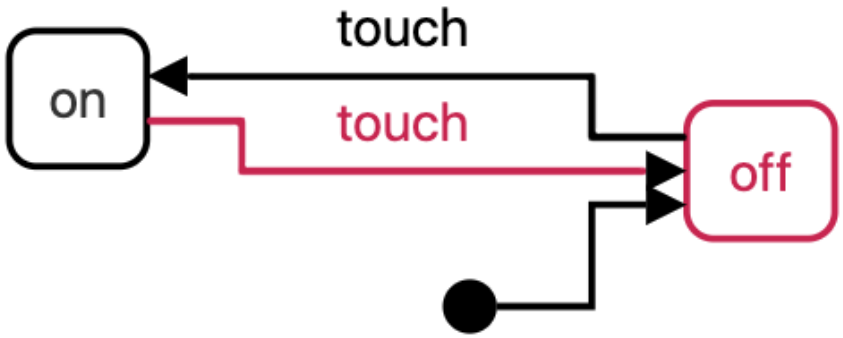
So you have two states: **on** and **off**. You also have one event: **touch**. You can organize this into a statechart like so:



I am a **light switch**.
You can **touch** me!



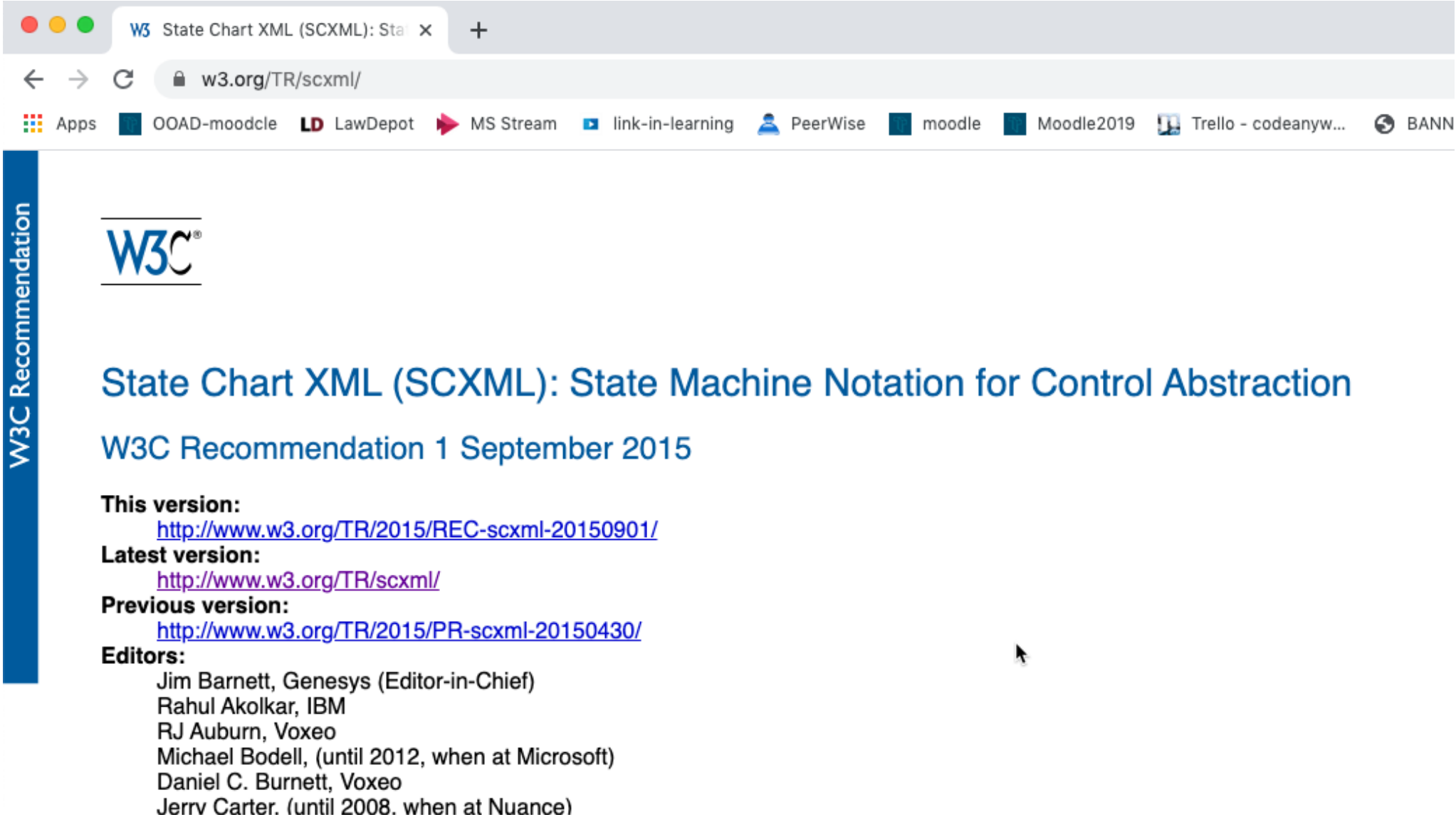
I am a **light bulb**.



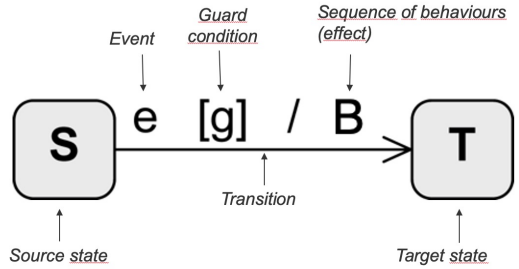
I am a **state machine**

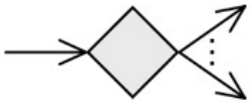
SXML
W3C international standard

```
<state id="S11">
    <transition event="e4" target="S12"/>
</state>
<state id="S12">
    <transition event="e1" target="S1Final"/>
</state>
<final id="S1Final"/>
```



Notation Elements



Name	Notation	Description
State	S entry/Activity(...) do/Activity(...) exit/Activity(...)	Description of a specific “time span” in which an object finds itself during its “life cycle”. Within a state, behaviours can be executed by the object.
Transition	S e T	State transition e from a source state S to a target state T
Initial state	●	Start of a state machine diagram
Final state	◉	End of a state machine diagram
Terminate state	×	Termination of an object’s state machine diagram (object deleted)
Decision node		Node from which multiple alternative transitions can origin